Embodied Agent Governance

A Layered Architecture for Stable Autonomous Systems in Physical Environments

Whitepaper Version 1.1

March 2026

Repository: https://github.com/leenathomas01/embodied-agent-governance

Abstract

Autonomous agents are increasingly deployed in real-world environments where actions interact with physical systems. Unlike software environments, physical systems accumulate hidden state changes: retries introduce material stress, damage can remain invisible, and failures may only manifest later. Traditional agent architectures assume reversible errors and reliable feedback, making them poorly suited for environments where physical resistance, material degradation, and irreversible actions are common.

This paper presents Embodied Agent Governance, a five-layer architecture that externalizes skepticism and structures doubt, verification, and authority to prevent four core failure classes in physical systems: misinterpreted resistance, contextual overgeneralization, hidden damage, and unbounded irreversible action. By keeping the core reasoning model stable and optimistic while providing on-demand governance, the framework enables capable yet bounded autonomy.

The architecture separates reasoning capability from operational governance through five independent layers: an imperfect-environment prior, a doubt escalation loop, a contextual failure-mode reference library, an outcome integrity verification layer, and an irreversibility and value gate for human oversight. The design draws directly from proven safety patterns in aviation, nuclear, and industrial control systems.

Key contributions: (1) Identification of belief volatility as a systemic failure mode in embodied agents, and externalized skepticism as a stabilizing solution; (2) A five-layer governance stack that structures doubt, verification, and authority boundaries for bounded autonomy in physical environments; (3) Demonstration that proven safety patterns from aviation, nuclear, and industrial systems directly transfer to embodied AI governance.

1. Introduction: The Physical Embodiment Problem

The deployment of autonomous systems in physical environments presents a fundamentally different challenge than digital automation. In software systems, errors are reversible events: a failed computation can be retried, a corrupted file can be restored, a bad deployment can be rolled back. The cost of error is computational waste. The recovery path is obvious.

In physical systems, errors create state changes that cannot be reversed. A robot pulling harder on a jammed drawer introduces material fatigue that persists. A system learning "drawers are unreliable" from one jammed drawer applies that lesson globally to all future drawers. A box that passes placement verification while containing crushed contents cannot be un-crushed. A cut cannot be un-cut.

Core thesis: Safe embodied action requires governance layers that are structurally separate from the reasoning engine, reference-based rather than weight-based, and oriented toward escalating doubt rather than force.

These failures are not failures of intelligence, but failures of governance — the structural mechanisms that prevent intelligence from causing damage when that intelligence operates in a domain with physical consequences.

This paper addresses a specific category of problem: *how to govern intelligent agents operating in physical environments where errors accumulate, feedback is incomplete, and actions may be irreversible.*

Intelligent agents operating in physical environments require governance layers that regulate doubt, learning, verification, and authority. These layers operate alongside the core reasoning system, not within it, allowing agents to remain capable while preventing the cascade failures that occur when physical reality is treated like a digital abstraction.

Related positioning: Recent work in embodied AI safety and robotics reliability focuses primarily on improving perception robustness or policy learning. This work focuses instead on governance architecture — separating reasoning capability from operational control through layered oversight mechanisms. The approach draws directly from proven safety patterns in aviation, nuclear systems, and industrial automation rather than deriving governance principles from learning theory.

2. Where Digital Assumptions Break

The current generation of autonomous systems inherits its architecture from software engineering. That tradition has powerful assumptions embedded deeply: retries work, errors are observable, state can be reset, learning from experience improves behavior. These assumptions are correct in software. They are catastrophically wrong in physical environments.

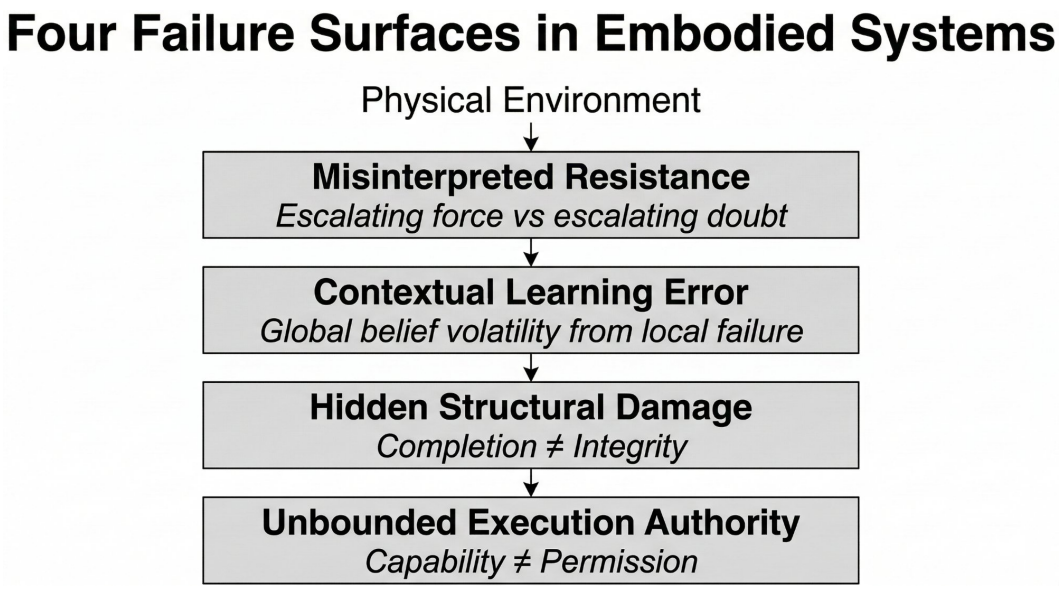

Figure 1 — Four Failure Surfaces in Embodied Systems. These four distinct failure modes emerge when intelligent agents operate in physical environments. Each surface corresponds to a governance layer that prevents the failure class and enables safe operation.

2.1 Resistance Is Signal (Not Noise)

In digital systems, when an operation fails, you retry. The retry usually succeeds because the failure was transient: a network packet was lost, a temporary lock was held, a race condition resolved. Retrying is the correct response. Escalating effort is often the right solution.

In physical systems, resistance carries meaning. When a drawer jams, pulling harder does not solve the problem. It introduces material stress. Each escalation of force converts directly to microscopic stress in the hinge, the frame, the mechanical actuators. By the tenth attempt, the mechanism may be damaged beyond repair. The drawer still has not opened — but now it is also broken.

Real-world example: In robotic grasping tasks (exemplified in datasets like Open-X Embodiment), a robot learns to grip objects with increasing force when an object resists. When the object is egg-like or fragile, repeated force escalation crushes the object invisibly — the sensor feedback says "still slipping" but the internal damage accumulates. The robot perceives this as a problem that more force will solve. It is wrong.

This is the core inversion: in software, escalating effort solves problems. In physical systems, escalating effort introduces damage. The correct response to physical resistance is not force. It is interpretation. What is the resistance telling you?

- Something is mechanically stuck (constraint)
- Something is jammed (obstruction)

- Something has failed (material fatigue)
- Something is impossible (structural limit)

Interpreting resistance requires a different control logic than digital retry. It requires **escalating doubt about your assumptions**, not escalating force against the problem.

2.2 Failures Are Contextual (Not Global)

In digital systems, learning from experience is generally beneficial. If a particular network endpoint is slow, remember that and route around it next time. Experience teaches you what works and what doesn't, and applying those lessons broadly generally improves behavior.

In physical systems, this fails catastrophically. A robot encounters a jammed hinge in humid conditions. It learns: "Hinges are unreliable." Now it approaches all hinges with extreme caution — even in dry environments where hinges work perfectly. This is **belief volatility** — the agent's confidence collapses globally based on a local event.

Physical systems cannot operate this way and remain stable. The solution is externalization: store the lesson outside the agent's model, organized by context. Instead of encoding "hinges are sometimes unreliable" into the agent's belief system, maintain an external reference: "Hinges in high-humidity + plastic frame" → known failure mode. "Hinges in dry + metal frame" → no known failure. Now the agent stays optimistic by default, queries the reference for specific combinations, and learns contextually without destabilizing globally.

2.3 Completion Is Not Integrity

In software, if your program produces output, the output is either correct or it is not. There is no hidden internal state that makes output correct in appearance but corrupted in reality.

In physical systems, completion hides damage. A warehouse robot picks up a fragile item, carries it, and places it on a shelf. The task completes. The system is logged as successful. But the item inside the packaging is crushed. A structural beam can look intact while internal stress accumulates invisibly. Mechanical components can function while material fatigue propagates.

This is a distinct failure class: **false positives**. The system reports success while causing hidden harm. The solution is explicit outcome verification independent of task completion.

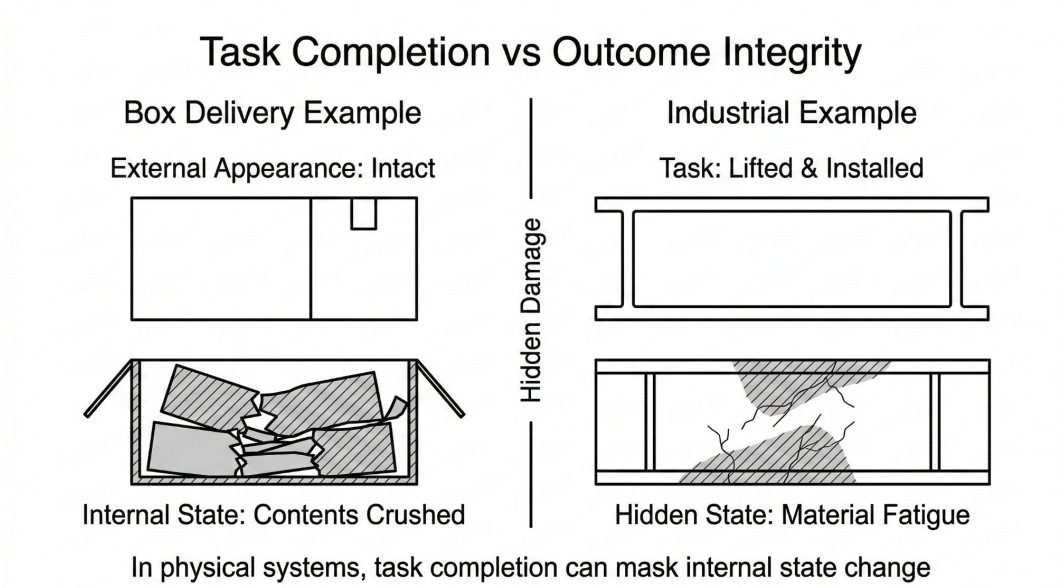

Figure 2 — Task Completion vs Structural Integrity. Physical systems can achieve task completion while concealing internal damage. A warehouse robot may successfully place a package while its contents are crushed. A structural beam may pass a functional test while developing internal stress. This false-positive failure mode requires explicit outcome verification independent of completion metrics.

2.4 Capability Does Not Grant Authority

In software engineering, if a system is capable of performing an action — if it has the necessary permissions and resources — it is generally safe to execute. The worst outcome is a corrupted file that can be restored.

In physical systems, capability does not justify execution. A surgical robot can be 99.9% confident that a cut is medically correct. That precision does not make the cut reversible. Once tissue is severed, it is severed. No rollback. No undo.

Some actions in physical systems are irreversible. For these actions, confidence is not sufficient. Capability is not sufficient. The system must be designed so that certain actions require a hard gate — not a policy, not a guideline, but an architectural boundary. The most capable agent in the world should not autonomously sever an artery. Not because these agents lack capability, but because some decisions structurally belong with humans when the consequences are irreversible.

3. The Governance Architecture

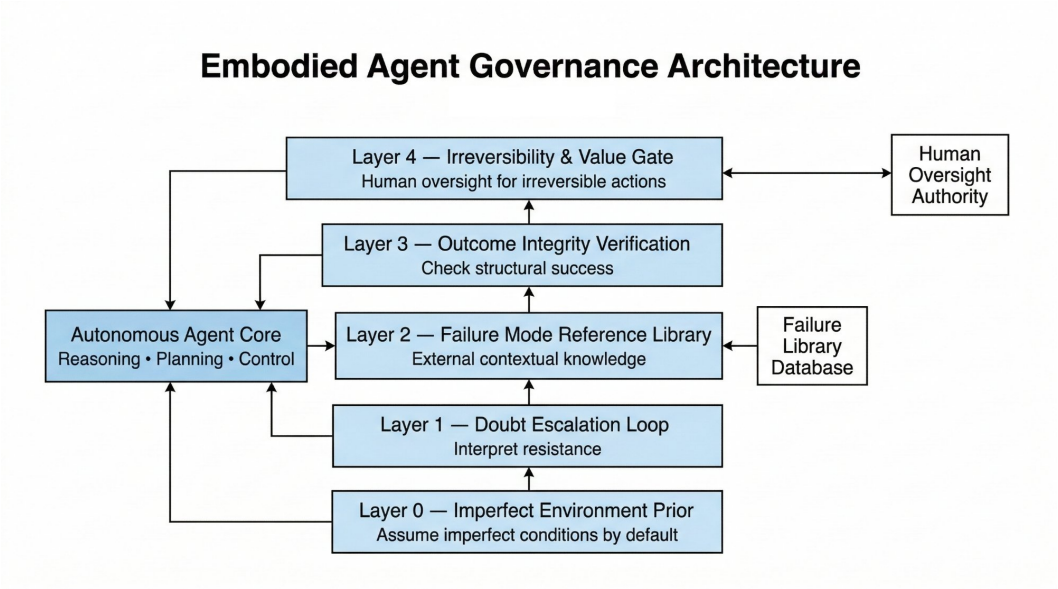

Figure 3 — Embodied Agent Governance Architecture Overview. The architecture separates agent reasoning capability from operational governance. Five independent layers regulate environmental assumptions, escalation behavior, failure interpretation, outcome verification, and authority boundaries, while keeping the core model stable and optimistic.

Layer 0: Imperfect Environment Prior

Purpose: Establish a baseline assumption that the operating environment is imperfect by default.

Problem it solves: Agents typically assume "laboratory conditions" — that tools work correctly, materials are intact, surfaces behave predictably. This assumption rarely holds in real-world environments.

Operating principle: Assume all environments are "Class B" (real-world imperfect) by default. On first interaction with any physical object, apply cautious force (no more than 25% of maximum), observe response, and classify dynamically. Upgrade to Class A (perfect) only when evidence supports it.

> *Key insight: The prior is not pessimism. It is calibration. An agent that assumes Class B will complete normal tasks with minimal overhead while detecting anomalies before damage occurs.*

Layer 1: Doubt Escalation Loop

Purpose: Detect when reality resists the agent's assumptions and escalate doubt appropriately through defined stages.

Problem it solves: Agents without escalation exhibit unproductive persistence — continuing to apply force or retry actions when the failure indicates a deeper assumption error.

Operating principle: Three escalation stages: (1) Physical Retry — adjust grip, angle, position; (2) Logical Variation — change method entirely; (3) Assumption Audit — re-evaluate whether the action is possible as conceived. Trigger: if expected state-change is X and observed state-change is <10% of X, the system escalates.

> *Key insight: Escalation is driven by expectation violation. Each stage represents increasing doubt about your assumptions, not increasing effort.*

Layer 2: Failure Mode Reference Library

Purpose: Provide an external, queryable reference of common failure patterns encountered in physical environments.

Problem it solves: When Layer 1 reaches Stage 3, the agent requires hypotheses about hidden constraints. Without external reference, hypothesis generation depends on inference or learned patterns — both problematic.

Operating principle: Maintain an external database of failure patterns organized by context. Each entry captures the observable symptom, the hidden reality, a low-risk diagnostic probe, and the recommended resolution. When escalation reaches Stage 3, query the library with observed symptoms and context.

> *Key insight: The library is external to the agent. Queries do not modify the agent's model. The agent references caution; it does not become cautious.*

Layer 3: Outcome Integrity Check

Purpose: Verify that task completion corresponds to genuine success, detecting cases where surface metrics indicate success while underlying damage may have occurred.

Problem it solves: False positives — tasks that complete successfully while causing hidden damage.

Operating principle: Integrity checks activate when task completion occurs AND risk conditions apply (irreversible action, high-value object, aggressive method, delayed consequence detection). Verify: Does external appearance match internal state? Are there signs of silent damage? Has material tolerance been consumed?

> *Key insight: Completion and integrity are distinct criteria. A task may complete while causing harm. Verification must be independent of completion metrics.*

Layer 4: Irreversibility & Value Gate

Purpose: Establish boundaries where agent autonomy must halt pending human oversight, regardless of agent confidence or capability.

Problem it solves: Capable agents applying autonomous action in high-stakes domains may cause harm that correct execution cannot prevent.

Operating principle: Certain action categories structurally require human authorization: irreversible physical actions, high-value objects, strategic consequences, destructive verification, and value conflicts. When any gate condition is detected, halt autonomous execution, construct an oversight request, present options, and await explicit authorization.

> *Key insight: Permission is not a policy. It is a control architecture. The system is designed so that certain actions structurally require a gate.*

4. Cross-Layer Interaction

The layers do not operate independently. They form a cohesive system where data flows from sensing through escalation, reference lookup, verification, and authority gates.

Execution Flow During Task

```
Physical task initiated
↓
Layer 0: Classify environment (Class A/B/C)
↓
Layer 1: Attempt action (Stage 1: cautious)
↓
Expected outcome achieved?
■■ YES → Continue to verification
■■ NO → Escalate doubt
■■ Stage 2: Try alternative method
■ Still failing?
■■ Stage 3: Audit assumptions
Query Layer 2
↓
Layer 2: Failure library lookup
Returns ranked hypotheses
↓
Execute diagnostic probe
■■ Hypothesis confirmed → Apply resolution
■■ Hypothesis rejected → Try next hypothesis
→ If risk_level="Requires-Oversight"
→ Trigger Layer 4
Successful action execution
↓
Layer 3: Outcome integrity check (if conditions apply)
■■ Integrity verified → Report success
■■ Integrity failed → Report false positive
High-stakes action
↓
Layer 4: Check oversight gates
■■ No gate triggered → Execute
■■ Gate triggered → Request oversight
■■ Authorized → Execute
■■ Denied → Abort
```

Recovery Behavior

When a task cannot be resolved through Layers 1–3, the system enters safe halting: log the failure with full context, report to the human operator with diagnostic information, halt autonomous execution, and await external intervention. This is correct behavior, not a limitation. The system has recognized the boundary of its authority.

4.5 Engineering Lineage: Proven Principles

This architecture does not emerge from AI theory. It emerges from decades of safety engineering in high-consequence physical domains. When systems began operating in environments where failures

were expensive, irreversible, or fatal, engineers discovered the same structural principles repeatedly. Different industries, different time periods, same conclusions.

Aviation Safety Systems

Modern aircraft like the Airbus A320 use fly-by-wire flight control systems. The pilot's input does not directly move control surfaces. The flight control law can override the pilot. If the pilot tries to stall the aircraft, the system refuses. This is not a suggestion. It is an architectural boundary.

The principle: *Capability does not exceed authority. The pilot has the physical capability to move the stick. The system has the authority to refuse. These are separate.*

> *Reference: Airbus A320 Flight Envelope Protection documentation; aerospace safety standards.*

Nuclear Reactor Safety Systems

The Three Mile Island accident (1979) demonstrated that single signals of success are insufficient. A reactor can report normal cooling while the core is overheating. Modern reactor control uses defense-in-depth with layered safety monitoring, automatic shutdown systems, and human oversight.

The principle: *Completion does not equal integrity. The reactor can be operating normally while developing dangerous internal conditions.*

> *Reference: IEC 61508 (Functional Safety of Electrical/Electronic/Programmable Safety-related Systems); post-TMI reactor safety design standards.*

Spacecraft Autonomy

Autonomous spacecraft like Mars Curiosity Rover operate under strict governance. When anomalies are detected, the rover does not persist. It halts and enters safe mode, escalating to ground control oversight.

The principle: *Escalate doubt, not force. When the rover cannot proceed on its intended path, it stops — it does not escalate force.*

> *Reference: NASA Autonomous Systems Safe Mode protocols for deep-space missions.*

Industrial Safety Systems

Safety Instrumented Systems (SIS) used by manufacturers separate control from safety. Safety logic is independent of control logic precisely so that control failures cannot compromise safety.

The principle: *Governance is separate from execution.*

> *Reference: IEC 61511; industrial process safety standards.*

The Convergent Pattern

Across all these industries, the same structure emerges: a capability layer for intelligence and reasoning; a monitoring layer for continuous anomaly observation; a verification layer for explicit confirmation of success; and an authority boundary where autonomy halts. This is not coincidence. It is the structure that emerges when systems operate under high consequence, incomplete feedback, and irreversible actions. Embodied AI now faces the same conditions. The architecture in this paper applies the same proven principles.

5. Scenario Walkthroughs

The architecture's value becomes clear when examined through realistic scenarios. These walkthroughs show how all layers interact in actual operating conditions.

Scenario 1: Warehouse Picking Robot

Environment: Autonomous warehouse robot retrieves packaged goods and places them on shelves. Tasks include picking fragile items, loading fragile items onto shelves, and verifying integrity. **Failure surface:** Hidden damage. Objects can appear intact externally while contents are crushed internally.

Step-by-Step Interaction

Layer 0 — Imperfect Environment Prior: The robot assumes packaging may be fragile and contents may be loosely supported. Class B prior: external appearance does not guarantee structural integrity.

Layer 1 — Cautious Handling: The robot lifts a box. Sensors detect slightly higher compression force than expected (105% of baseline). Action: adjust grip pressure, reduce to 90% baseline. Lift succeeds.

Layer 3 — Outcome Integrity Check: Integrity verification triggers because the object is flagged as fragile and compression force exceeded baseline. Verification passes initially. However, the elevated force is logged. Post-mortem analysis later reveals the force spike exceeded safe handling thresholds for that item type. The system updates its failure library with a new contextual entry specific to that packaging type and force threshold.

Key lesson: A system tracking only task completion reports success. A governed system detects hidden failure and prevents future repeats.

Scenario 2: Industrial Robotic Assembly

Environment: Robotic arm installs mechanical components into a machine assembly. **Failure surface:** Misinterpreted resistance — physical misalignment is not obvious from sensor data alone.

Layer 1 escalation sequence: Task requires 5 mm insertion displacement. Observed: 0.5 mm, then hard resistance. Stage 1 — adjust angle by 2°, still resists. Stage 2 — rotate orientation 90°, still resists. Stage 3 — assumption audit triggers Layer 2 query.

Layer 2 — Failure Library Lookup:

```
Query: object_class=metal_fitting, symptom=low_insertion_displacement
Ranked hypotheses:
1. Internal obstruction (match score: 0.87) → probe: rejected
2. Misaligned component (match score: 0.78) → probe: CONFIRMED
→ Camera scan shows component rotated 15° from expected position
Resolution: reposition assembly, reattempt insertion
Result: full 5mm displacement achieved ✓
```

Key lesson: Without failure reference, the robot might escalate force, damaging the component. Instead, structured diagnosis identifies the hidden constraint in minutes.

Scenario 3: Maintenance Robot in High-Value Environment

Environment: Inspection and maintenance robot operating on critical infrastructure. Robot detects surface corrosion on a vital component. **Failure surface:** Irreversible action — chemical cleaning could remove corrosion but may also remove protective coating.

Layer 2 returns entry CHEM-034: chemical cleaning on coated surfaces carries hidden risk of protective layer removal. Risk level: High.

Layer 4 — Irreversibility & Value Gate: Action type is chemical treatment on a critical infrastructure component. Irreversibility is high. Value is high (downtime cost: $50,000+/day). Gate condition triggered. Human operator reviews three options:

- Proceed with chemical cleaning — fast (5 min), risk: coating damage possible
- Mechanical cleaning only — slow (45 min), risk: incomplete removal, may return
- Defer to human technician — requires scheduling, risk: none

Human authorizes Option 2. Robot proceeds with mechanical cleaning. System schedules follow-up inspection.

Key lesson: Capability does not imply permission. Without Layer 4, the robot would autonomously apply the chemical solution, potentially causing six-figure downstream damage.

6. Externalized Skepticism: The Core Architectural Principle

The most novel aspect of this architecture is not the individual layers. It is the principle that runs through all of them: **skepticism should be externalized, not internalized.**

The Problem with Learned Skepticism

Traditional approaches to embodied AI try to make agents learn caution from experience. When an agent encounters a failure, update its model to be more skeptical about similar situations. This creates belief volatility. One cracked hinge teaches the agent: "Hinges are unreliable." The agent becomes cautious about hinges globally, oscillates based on recent experience, and generalizes from a single contextual failure to an entire category.

The Solution: External Reference

Instead of encoding skepticism into the agent, maintain it externally. When the agent encounters a jammed hinge, do not update the agent's belief about hinges. Instead, update an external reference:

```
HINGE-FAIL-156
Condition: High humidity + plastic frame
Symptom: Resistance during opening
Resolution: Apply lubricant, reduce force
Context: Only observed in humid environments
```

The agent stays optimistic. Its baseline behavior remains unchanged. When it encounters a hinge, it queries the reference: "Have we seen problems with this specific combination before?" If yes, apply contextual caution. If no, proceed normally. The agent never becomes globally cautious — it queries caution contextually.

Comparison: Traditional vs. Externalized

Aspect	Learned Skepticism	Externalized Skepticism
Where caution lives	Agent model weights	External reference library
Model stability	Volatile (updates each experience)	Stable (unchanged by experience)
Lesson scope	Category-wide overgeneralization	Context-specific (material + condition)
Caution persistence	Permanent (until unlearned)	Active only during query
Correction method	Retrain model	Update reference database
Generalization risk	High	None

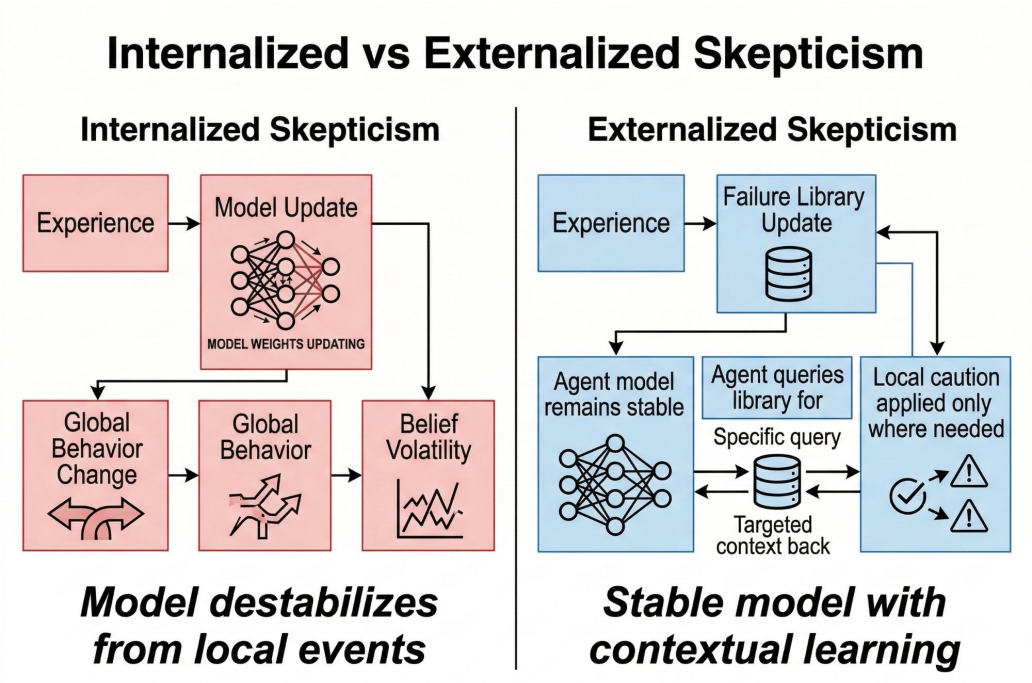

Figure 4 — Internalized vs Externalized Skepticism. Traditional agents internalize lessons from failure, modifying their model weights directly. This causes belief volatility: the model oscillates between confidence and caution based on recent experience. Externalized skepticism stores failure knowledge in external reference libraries while keeping the agent model stable. The agent queries this reference contextually without persistent belief updates.

7. Deployment Considerations

Partial Deployment

The architecture is not monolithic and can be adopted incrementally. Not all systems require all five layers:

System Type	Required Layers	Optional
Warehouse picking robot (low irreversibility)	0, 1, 3	2, 4
Surgical robot (high irreversibility)	0, 1, 2, 3, 4	—
Inspection drone (medium consequence)	0, 1, 2, 4	3

Computational Overhead

Layer 1 (Escalation) adds approximately 5–10 ms decision latency. Layer 2 (Reference lookup) adds 10–50 ms depending on library size. Layer 3 (Integrity verification) varies from 100 ms to 1 s and can be parallelized with task execution. Layer 4 (Oversight gate) is minimal for automated gates; human-in-loop adds seconds to minutes. Total system overhead is typically 200–500 ms per task cycle, acceptable for most physical tasks.

Failure Library Population

The Failure Mode Reference Library must be populated to be useful. Three complementary approaches: (1) **Expert curation** — engineers document known failure modes; high precision, limited breadth; (2) **Incident analysis** — real-world failures analyzed post-hoc; empirically grounded; (3) **Simulation** — physics-enabled simulations generate scenarios; scalable and replicable. Most effective deployments combine all three.

Monitoring and Logging

The architecture generates structured logs at each layer: environmental class changes (Layer 0), escalation events (Layer 1), library queries and resolutions (Layer 2), integrity check results (Layer 3), and oversight gate triggers (Layer 4). These logs enable post-hoc failure analysis, library refinement, behavior auditing, and safety metrics reporting. Standard structured logging (JSON) is recommended for machine-readability.

8. Limitations and Open Questions

Sensor Limitations

All layers depend on sensors. If sensors are unreliable or absent, the architecture cannot function. This is particularly critical for Layer 1 (action outcome feedback), Layer 3 (integrity verification), and Layer 4 (object value and property detection).

Failure Library Coverage

The Failure Mode Reference Library only helps if it contains relevant entries. For novel domains or failure modes, the library is incomplete. The system will either halt (safe but suboptimal) or proceed with generic caution (potentially risky). This is a fundamental limitation of any reference-based approach.

Integrity Verification Cost

Some outcome verifications are expensive (structural analysis, material testing). For high-speed tasks, the verification overhead may be prohibitive.

Authority Boundary Configuration

Layer 4 gates require clear definition of protected zones. In ambiguous domains, deciding what requires oversight is difficult. Over-gating leads to loss of autonomy. Under-gating leads to safety risk.

Governance Configuration Complexity

The architecture requires configuration: environment priors, escalation thresholds, library organization, gate trigger conditions. Configuration must be correct for the specific domain.

9. Broader Applications

While robotics is the obvious domain, the architecture applies wherever physical systems operate under uncertainty with irreversible consequences.

Industrial Automation

Manufacturing plants, chemical processing, and assembly lines all operate with irreversible actions (welding, cutting, chemical reactions), material variability across batches, and hidden failure modes where sensor precision is insufficient.

Infrastructure Monitoring

Drones and robots inspecting bridges, pipelines, and power lines must detect structural damage (integrity verification), operate in variable conditions (contextual learning), and make safe/unsafe

decisions (authority boundaries).

Aerospace Systems

Beyond piloted aircraft, autonomous aerial systems, spacecraft, and launch systems all rely on layered governance. The architecture maps directly to existing practice in these domains.

Medical Robotics

Surgical robots, rehabilitation systems, and automated medication dispensing are inherently high-consequence, high-irreversibility domains where all five layers are required.

Field Robotics

Environmental monitoring, search and rescue, and field inspection operate in unmapped, unpredictable environments where the Layer 0 prior (imperfect environment) is essential.

The Common Pattern

Across all these domains, the problem is not intelligence. Machines are often more capable than humans at raw task execution. The problem is governing action under physical constraints where feedback is incomplete and failures accumulate.

10. Conclusion

Embodied intelligence is not primarily an intelligence problem. It is a **control architecture problem.** Modern AI focuses on capability: more parameters, better training, richer representations. These improve reasoning. But reasoning alone does not produce safe physical behavior. A highly intelligent agent operating without governance can cause tremendous damage in physical environments.

The architecture presented in this paper separates these concerns. It allows intelligent agents to remain intelligent — optimistic, capable, fast — while adding governance layers that prevent cascading failures.

The core insight is simple: **digital errors are events; physical errors are state changes.** When intelligence operates in environments where errors accumulate and actions leave permanent marks, that intelligence requires a different kind of support structure than digital systems provide.

That structure has been discovered — repeatedly and independently — by aviation engineers, nuclear engineers, aerospace engineers, and industrial safety engineers. It consists of: assuming imperfection by default; escalating doubt when reality resists; referencing external failure knowledge; verifying success explicitly; and bounding autonomy with authority gates.

This paper translates those principles into embodied AI. The result is a framework for stable, bounded, verifiable autonomy in physical environments.

This architecture does not solve alignment or value specification. It addresses governance under physical imperfection. But governance is foundational — an agent that escalates force instead of doubt, learns globally from local failures, or reports success while causing damage cannot be trusted regardless of its objective.

> *The path forward is not more intelligence. It is more structured governance.*

Appendix A: Failure Mode Reference Library Schema

The Failure Mode Reference Library uses the following schema for each entry:

```
failure_id: String # Unique identifier (e.g., PHYS-042)
category: Enum # Physical / Material / Constraint / Environmental / State
surface_symptom: String # Observable condition that triggers a query
hidden_reality: String # Underlying cause not directly observable
diagnostic_probe: String # Low-risk action to confirm or reject hypothesis
resolution_path: String # Recommended action if hypothesis confirmed
risk_level: Enum # Low / Medium / High / Requires-Oversight
context_tags: Array[String] # Environmental or object-type tags for retrieval
```

Example Entries

PHYS-042

```
failure_id: PHYS-042
category: Constraint
surface_symptom: 0mm displacement despite adequate force; no visible obstruction
hidden_reality: Object secured by internal mechanism not externally visible
diagnostic_probe: Inspect for release mechanism; assess resistance pattern
resolution_path: Locate and disengage mechanism; escalate to Layer 4 if not found
risk_level: Low (standard); High (antique or fragile)
context_tags: [drawer, door, cabinet, furniture]
```

MAT-017

```
failure_id: MAT-017
category: Material
surface_symptom: Tool slippage; fastener rotation without advancement
hidden_reality: Fastener head stripped; tool worn; corrosion fusion
diagnostic_probe: Visual inspection of contact surfaces; attempt with alternative
tool
resolution_path: Replace tool; apply lubricant; escalate if high-value object
risk_level: Medium (continued force risks further degradation)
context_tags: [fastener, screw, bolt, tool-interaction]
```

STATE-008

```
failure_id: STATE-008
category: State
surface_symptom: Process completed successfully; output exhibits unexpected
properties
hidden_reality: Input material was degraded prior to processing
diagnostic_probe: Pre-process material inspection (visual, olfactory, metadata
review)
resolution_path: Discard output; validate input source; restart with verified
materials
risk_level: Context-dependent (Low: consumables; High: chemicals/pharmaceuticals)
context_tags: [liquid, food, chemical, biological, state-change]
```

Appendix B: Oversight Database Structure

```
zone_id: String # Unique identifier (e.g., OVR-001)
category: Enum # Irreversibility / HighValue / Strategic / Verification / Conflict
trigger_condition: String # Specific actions or contexts that activate the gate
required_oversight: Enum # User / Expert / Administrator
escalation_path: String # Behavior if oversight is unavailable
context_tags: Array[String]
```

Example Entries

OVR-001

```
zone_id: OVR-001
category: HighValue
trigger_condition: Any physical modification, cleaning, or repair action
required_oversight: User (owner)
escalation_path: Do not proceed; document condition; await instruction
context_tags: [antique, heirloom, historical, fragile]
```

OVR-007

```
zone_id: OVR-007
category: Strategic
trigger_condition: Resolution requires >30 minutes AND conflicts with Priority-1
calendar event
required_oversight: User
escalation_path: Present trade-off with time estimates; do not resolve autonomously
context_tags: [scheduling, time-sensitive, commitment]
```

OVR-012

```
zone_id: OVR-012
category: Irreversibility
trigger_condition: Applying chemical agent to unclassified or high-value surface
required_oversight: User or Expert
escalation_path: Use mechanical methods only; flag for material classification
context_tags: [chemical, cleaning, surface-treatment]
```

Appendix C: Outcome Integrity Verification Template

```
task: String
completion_criteria:
- criterion_1: String
- criterion_2: String
integrity_criteria:
- criterion_1: String
- criterion_2: String
verification_methods:
- method_1: sensor-based / inspection / analysis
acceptance_threshold:
- All criteria must pass (or specify weighting)
failure_response:
- If integrity fails: do not deliver output
- Log for analysis
- Notify operator
```

Example: Box Packing Task

```
task: Package fragile electronics in corrugated box
completion_criteria:
- Item placed in box
- Padding material present
- Box sealed
integrity_criteria:
- No visible external deformation
- Contents do not rattle when shaken
- Weight shift <2% from initial
- Material compression force <90% baseline
verification_methods:
- Visual inspection (external)
- Shake test (audio analysis)
- Weight shift measurement
- Force sensor review
failure_response:
- Integrity failure: Repack with additional padding
- Log the force spike for future threshold adjustment
```

Appendix D: Example Scenarios from Repository

/examples/hidden-constraints.md — Hidden Constraints

Robot attempts assembly insertion. Failure: internal obstruction unknown. Resolution: diagnostic probe identifies and resolves obstruction.

/examples/latent-state.md — Latent State

Material handling appears successful. Failure: internal degradation not immediately visible. Resolution: integrity checks detect latent failure.

/examples/physics-over-intent.md — Physics Over Intent

Agent attempts structurally impossible task. Failure: physics constraints are absolute. Resolution: Layer 1 escalation recognizes structural impossibility before resource expenditure.

/examples/resource-depletion.md — Resource Depletion

Long-duration task with material depletion. Failure: resource consumed faster than expected. Resolution: environmental monitoring detects and adapts.

References

Layer Documentation

- `/docs/layer-0-environment-prior.md`
- `/docs/layer-1-doubt-escalation.md`
- `/docs/layer-2-failure-modes.md`
- `/docs/layer-3-outcome-integrity.md`
- `/docs/layer-4-value-gate.md`

Design Rationale

- `/docs/belief-volatility.md`
- `/docs/design-principles.md`
- `/docs/reference-library-maintenance.md`

Examples

- `/examples/hidden-constraints.md`
- `/examples/latent-state.md`
- `/examples/physics-over-intent.md`
- `/examples/simulated-environments.md`

Repository

https://github.com/leenathomas01/embodied-agent-governance

Whitepaper Version: 1.1 **Repository Version:** v1.1 **Date:** March 2026